Sistema de Gerontotecnología “Nonófono”

Passarelli Agustin Ezequiel, Mercaich Ignacio Gabriel, Muñoz Palazzo Ezequiel,
Mola Claudio Tomas
42825989, 42591320, 46700923, 34213995
Lunes Noche, Grupo L3

Resumen. El "Nonófono" actúa como sistema de asistencia para mayores, basado en la integración de hardware (ESP32) y software (Android/MQTT/FreeRTOS). El proyecto gestiona alertas, archivos multimedia y comunicación bidireccional, utilizando una arquitectura desacoplada.

Palabras claves: IoT, API Android, ESP32, FreeRTOS, MQTT, Telegram, Gerontotecnología.

1 Introducción

El "Nonófono" surge como una solución tecnológica para proveer asistencia a mayores mediante una aplicación Android, hardware ESP32 y protocolos de comunicación rápida. La aplicación gestiona notificaciones, alertas de emergencia y telemetría, centralizando la interacción entre usuarios y familiares.

El objetivo principal es asistir al adulto mayor mediante una interfaz simplificada que le permite:

- Enviar mensajes de texto predefinidos
- Enviar mensajes de voz de hasta un minuto
- Alertar a todos sus contactos ante una emergencia

Interfaz gráfica

Para ilustrar la interacción del usuario final con el sistema, se diseñó una interfaz gráfica intuitiva y modular. La **Figura 1** expone la vista principal de la aplicación (*MainActivity*), la cual actúa como panel de control central y provee retroalimentación visual en tiempo real sobre el estado de conexión con el servidor MQTT y la API de mensajería. A partir de este nodo central, el flujo de navegación se ramifica hacia módulos operativos específicos: la **Figura 2** detalla la interfaz de gestión, donde el administrador añade y vincula los números de los familiares (*Agregar Contactos*); mientras que la **Figura 3** muestra el módulo de emergencia utilizado para rastrear el hardware (*Localizar Dispositivo*). Finalmente, para evidenciar la resiliencia del

sistema y su tolerancia a cierres accidentales, la **Figura 4** exhibe la notificación persistente del *Foreground Service*, la cual garantiza que el motor de red continúe procesando eventos de forma asíncrona en segundo plano.



Figura 1. Interfaz principal y monitor de estado de red



Figura 2. Interfaz para el agregado y eliminado de contactos.

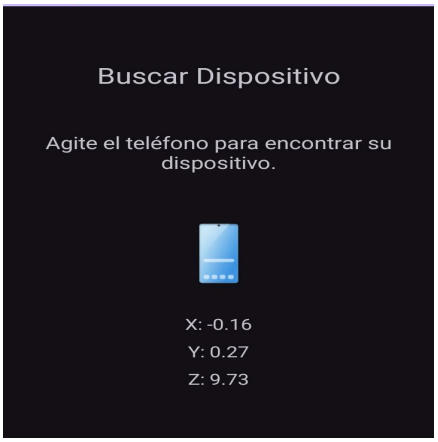


Figura 3. Pantalla para localizar el dispositivo



Figura 4. Notificación persistente del servicio de segundo plano.

2 Desarrollo

- **Repositorio GitHub:** https://github.com/UNLAM-SOA/2026-SOA-Q1-L3

Diagrama funcional y Arquitectura: El sistema separa la presentación de la lógica de negocio. MainActivity controla el estado de red, AgregarContactoActivity gestiona el directorio, y LocalizarDispositivoActivity rastrea la ubicación. La lógica reside en ContactosManager, mientras que NonofonoService (Foreground Service) gestiona el MQTTManager (Singleton) y la comunicación vía BroadcastReceivers. El ESP32 utiliza FreeRTOS, delegando el procesamiento de periféricos a tareas específicas (taskEvento).

Para facilitar un mayor entendimiento de la arquitectura diseñada, en la **Figura 5** referenciamos el circuito construído en el simulador Wokwi, en la **Figura 6** mostraremos el Diagrama de Estados que determinará el actuar de nuestra FSM, y en los diagramas de Secuencias de la **Figura 7** y **Figura 8** mostraremos el flujo del uso principal del proyecto y la secuencia de mensajes enviados y recibidos en MQTT, respectivamente.

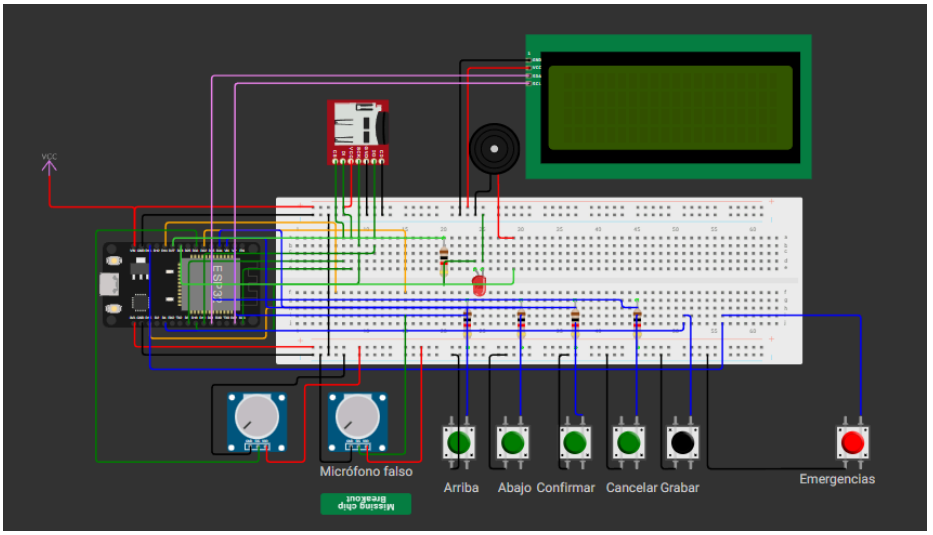


Figura 5. Esquema del circuito físico (Wokwi).

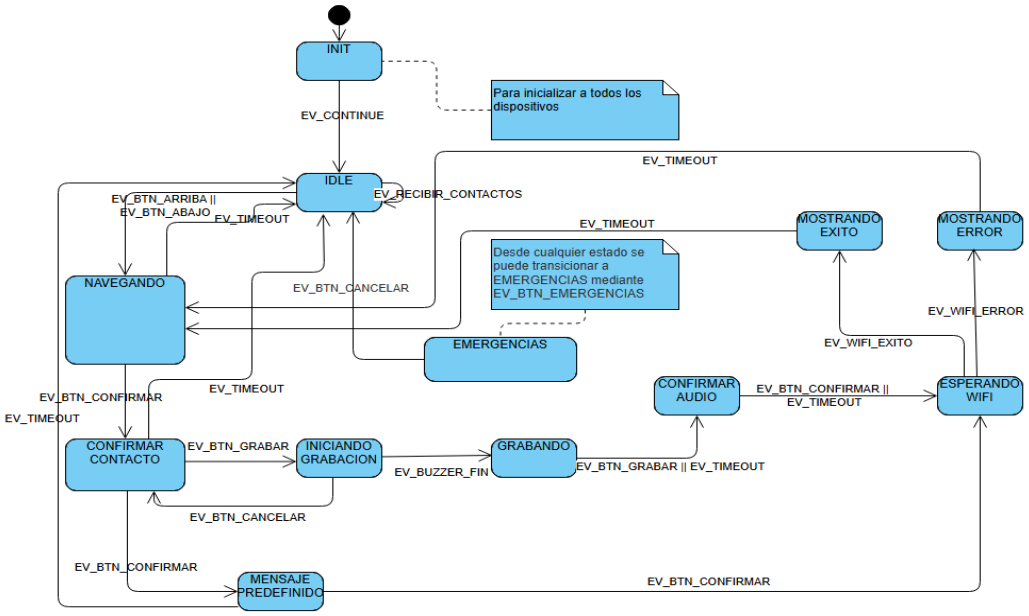


Figura 6. Máquina de Estados utilizada en el embebido.

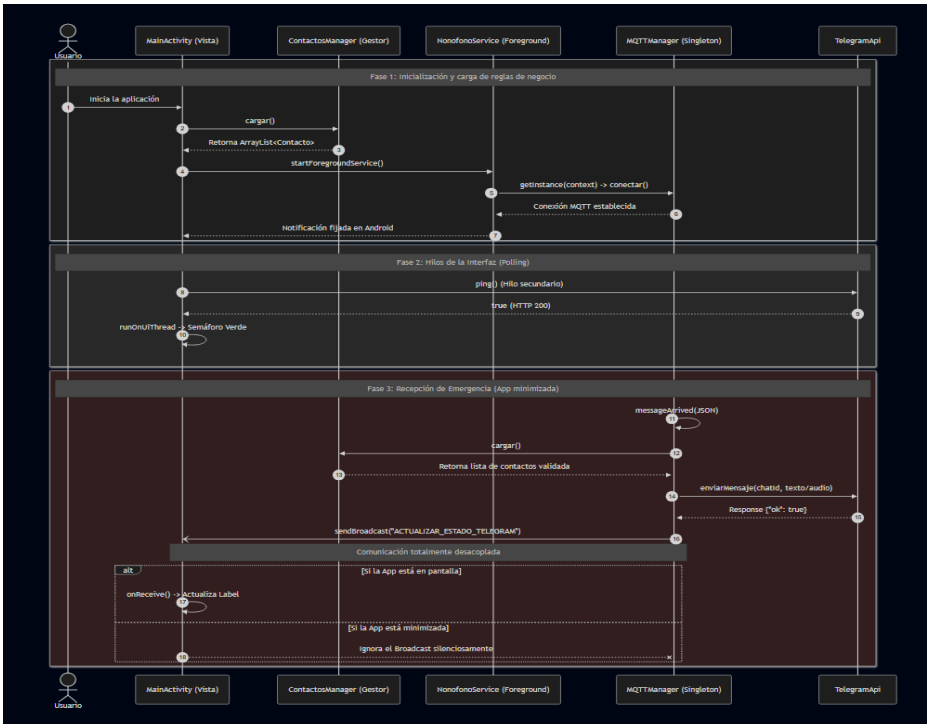


Figura 7. Flujo principal del uso de la aplicación en alto nivel

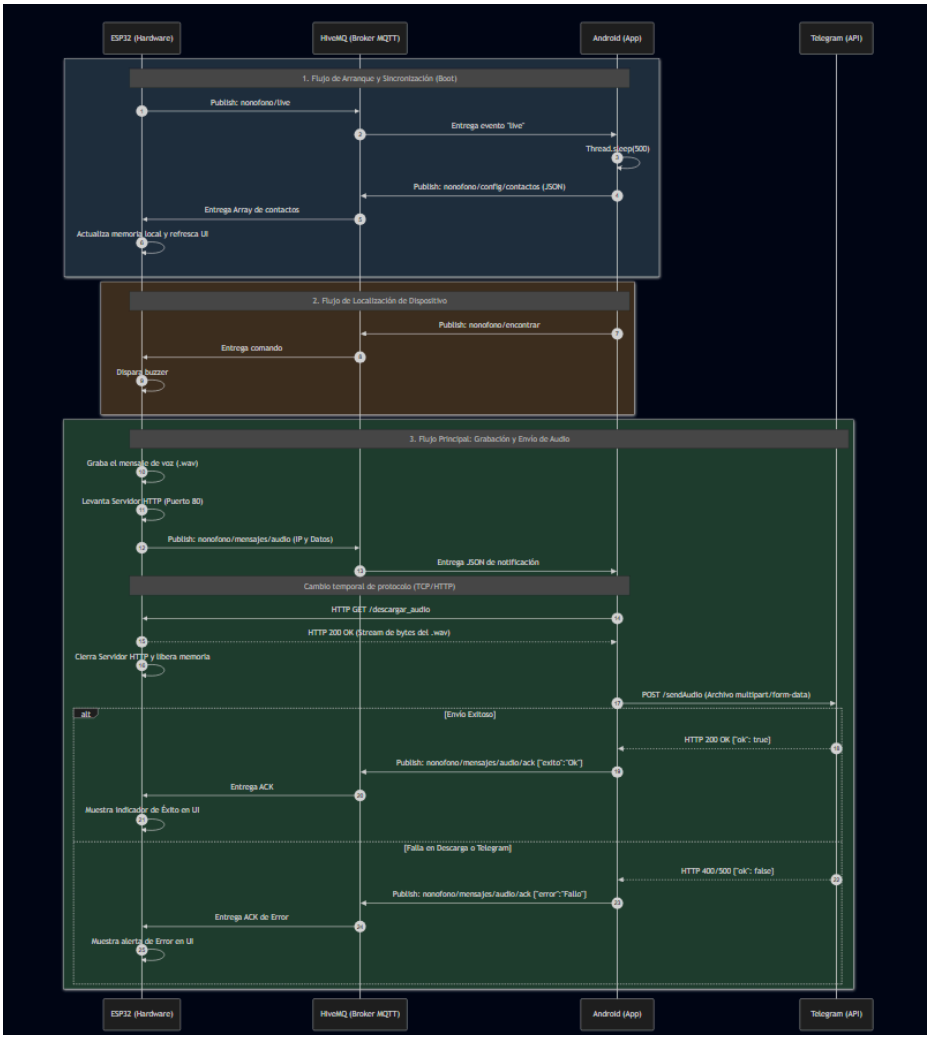


Figura 8: Secuencia de mensajes enviados y recibidos mediante MQTT

Integración con Telegram

La gestión de notificaciones hacia el entorno familiar se delega en la plataforma Telegram, operando mediante su API REST oficial para Bots. Bajo este esquema, la aplicación móvil funciona como un *gateway* o puente tecnológico, encargándose de transformar los eventos asíncronos recibidos desde el hardware (vía MQTT) en solicitudes HTTP seguras hacia la infraestructura de mensajería.

Este proceso de integración se divide operativamente en dos fases críticas: la vinculación de identidades y el despacho de alertas.

1. Vinculación y Gestión de Identidades (Inbound): Debido a que la API requiere un identificador único (*chat_id*) para el direccionamiento, se emplea un mecanismo de descubrimiento activo. Utilizando *Long Polling*, un hilo secundario consulta periódicamente el *endpoint /getUpdates*. Al recibir el número telefónico de un familiar, el sistema procesa el *payload* JSON. Cabe resaltar que esta lógica es delegada íntegramente al *ContactosManager*, el cual valida los datos contra el directorio local y persiste el identificador, manteniendo un estricto desacoplamiento de la interfaz visual.

2. Emisión de Alertas y Multimedia (Outbound): Con las identidades establecidas, el sistema actúa de forma unidireccional ante emergencias, permitiendo dos tipos de interacciones:

- **Notificaciones de Texto:** Al detectar una pulsación en el hardware, el cliente MQTT invoca el método */sendMessage*. El gestor recorre la lista de contactos autorizados y efectúa los despachos HTTP GET para asegurar la recepción en tiempo real.
- **Transferencia de Audio:** El envío de voz plantea desafíos por las limitaciones de memoria del embebido. Ante un nuevo mensaje, la aplicación realiza un *switch* temporal solicitando el archivo *.wav* mediante HTTP local al servidor del ESP32. Tras almacenarlo localmente, se construye una petición POST hacia */sendAudio*, encapsulando los datos bajo el estándar *multipart/form-data*.

Dicha arquitectura libera al microcontrolador de la gestión de cifrado TLS/SSL y del enrutamiento complejo, delegando el procesamiento intensivo y el manejo de errores de red al procesador del dispositivo móvil.

Enlace al repositorio de Github

Enlace al repositorio de Github

Manual de Usuario

El Manual de Usuario se encuentra adjunto en la entrega del documento.

3 Conclusiones

A continuación se presentan los resultados y reflexiones finales del proyecto:

Recaudos para un sistema tolerante a fallos

Para garantizar la disponibilidad continua del sistema, se adoptó una arquitectura resiliente operando en múltiples capas. En el entorno móvil, la designación del servicio en segundo plano con la directiva START_STICKY instruye al sistema operativo a restaurar el proceso automáticamente frente a recolecciones de memoria agresivas. A nivel de red, se implementó una recolección de recursos proactiva mediante el cierre explícito de descriptores de sockets y la aniquilación de instancias previas a cada reconexión, previniendo fugas de memoria que derivan en estados fantasma. Asimismo, la imposición de timeouts estrictos en las transacciones HTTP e hilos de consulta protege al sistema de bloqueos perpetuos, asegurando que los subprocesos se liberen correctamente incluso en escenarios de latencia extrema o pérdida súbita de conectividad por parte del nodo sensor.

Problemas encontrados y resueltos

La fase de integración expuso desafíos críticos de concurrencia y gestión de recursos compartidos. En el dispositivo móvil, la contención del hilo principal ocasionada por el bloqueo transaccional del Long Polling se rectificó mediante un esquema asíncrono de paso de mensajes utilizando BroadcastReceivers. En el contexto del microcontrolador, se observó que la transmisión continua de grandes buffers HTTP provocaba la inanición de las tareas subyacentes del RTOS, resultando en desconexiones abruptas del cliente MQTT. Esta saturación de red se resolvió calibrando el planificador mediante la inyección estratégica de retardos, lo cual permitió ceder ciclos de reloj cruciales para que las colas de eventos de hardware y la pila TCP/IP interna pudieran despachar las interrupciones de radiofrecuencia sin colapsar.

Lecciones aprendidas

El desarrollo del proyecto evidenció el valor técnico de aplicar pautas de Arquitectura Limpia y principios SOLID en ecosistemas distribuidos. Quedó demostrada la importancia de evitar la sobrecarga de los controladores de la interfaz gráfica, comprobando que delegar el procesamiento de las reglas de negocio y validaciones estrictamente a una clase *Gestor* facilita la mantenibilidad y previene fallos de concurrencia. Paralelamente, la experiencia subrayó las limitaciones de cómputo inherentes a los sistemas embebidos; frente a la ejecución de tareas intensivas, se reafirmó que el microcontrolador opera con mayor estabilidad actuando como un nodo perimetral de telemetría ligera, delegando el peso de las integraciones complejas y el consumo de APIs de terceros hacia el dispositivo móvil, el cual posee la holgura de recursos necesaria para gestionar dichas cargas.

4 Referencias

1. Banks, A., Gupta, R.: MQTT Version 3.1.1. OASIS Standard. (2014)
2. FreeRTOS Real Time Kernel (RTOS). Amazon Web Services.
3. Android Developers: Foreground Services. Google.
4. Telegram Bot API Documentation. Telegram FZ-LLC.
5. Martin, R. C.: Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, Boston (2017)